UCR-MFG: Uncertainty-Conditioned Relational Routing for Multi-Scale Fuzzy Graph Object Detection

Technical report: Introduction, Methodology and Working Guide

Summary. UCR-MFG extends the multi-scale fuzzy graph detector MFG-YOLO [6] with four coordinated contributions: (C1) dual predictive uncertainty that separates aleatoric from epistemic uncertainty; (C2) a router that uses this uncertainty, together with node features, fuzzy affinities and geometry, to keep only useful graph relations; (C3) benefit-aware multi-scale message passing that computes and weighs only routed messages; and (C4) an uncertainty-aware gate that fuses visual and graph-enhanced features by their reliability. Detections are returned with class, box, box standard deviation, aleatoric and epistemic uncertainty. This report states the problem, explains in plain language how the system works, gives the full formulation, losses, training curriculum and algorithms as implemented, and ends with a practical guide to running the project.

1 Introduction

Object detection locates and classifies the objects in an image, and underpins autonomous driving, surveillance, drone-based monitoring and industrial inspection. Deep detectors, YOLO-style architectures in particular, have pushed accuracy and speed far, and recent work adds multi-scale features, attention, graph neural networks and fuzzy representations to handle visually complex scenes. Detection in cluttered, changing environments nevertheless remains unreliable: models fail when objects are partly hidden, have blurred boundaries, appear at very different scales, or are captured under shifting lighting, weather, camera or location. Aerial and UAV imagery combines all of these with dense clusters of very small objects.

Uncertainty is a second weakness. Most detectors emit a single confidence score and do not separate uncertainty caused by missing information in the input (*aleatoric*) from uncertainty caused by the model's limited knowledge (*epistemic*) [11]. Graph-based detectors add their own problems: they introduce unnecessary connections that raise cost, and every retained edge passes a message whether or not it helps.

Existing fuzzy graph detectors fall short in four ways. (i) Feature-level relational uncertainty is not paired with separate aleatoric and epistemic estimates. (ii) Fuzzy graph construction measures how strongly two regions are related, but not whether passing information along that relation will help detection. (iii) Message passing does not weigh what each message is worth against what it costs. (iv) Fusion does not account for the relative reliability of the visual and graph-enhanced representations. When a region is occluded, blurred or ambiguous its own features may not support a reliable decision; the model should determine which relations actually help that uncertain region, adapt information transfer across scales, and trust each feature source according to its reliability.

Related work supplies each ingredient separately. Deformable DETR [2] samples a sparse set of multi-scale points, but from appearance alone, with no notion of the reliability of the sampled evidence. RT-DETR [5] decouples intra-scale interaction from cross-scale fusion for real-time detection, but fuses scales with fixed learned weights. GraphFPN [3] passes context over a feature-pyramid graph with deterministic similarity edges. Deterministic uncertainty [4] reads uncertainty from feature space in one pass, without splitting it. MFG-YOLO [6] models region affinity with a heavy-tailed Student-t kernel so ambiguous pairs keep a small non-zero link, but its graph is fixed once built and its uncertainty is not decomposed. UCR-MFG closes this gap with four contributions:

- **C1 Dual predictive uncertainty modeling:** aleatoric and epistemic uncertainty per location and per detection.
- **C2 Uncertainty-conditioned relational routing:** candidate relations are scored from node features, fuzzy affinity, both uncertainties and geometry; only the most useful are kept, with larger budgets for more uncertain locations.

- **C3 Benefit-aware multi-scale message passing:** messages flow only along routed edges within and across scales, each scaled by a learned benefit gate.
- **C4 Uncertainty-aware feature fusion and probabilistic detection:** a gate conditioned on features and uncertainty mixes visual and graph features; the head outputs class, box and uncertainty.

2 How UCR-MFG works: the big picture

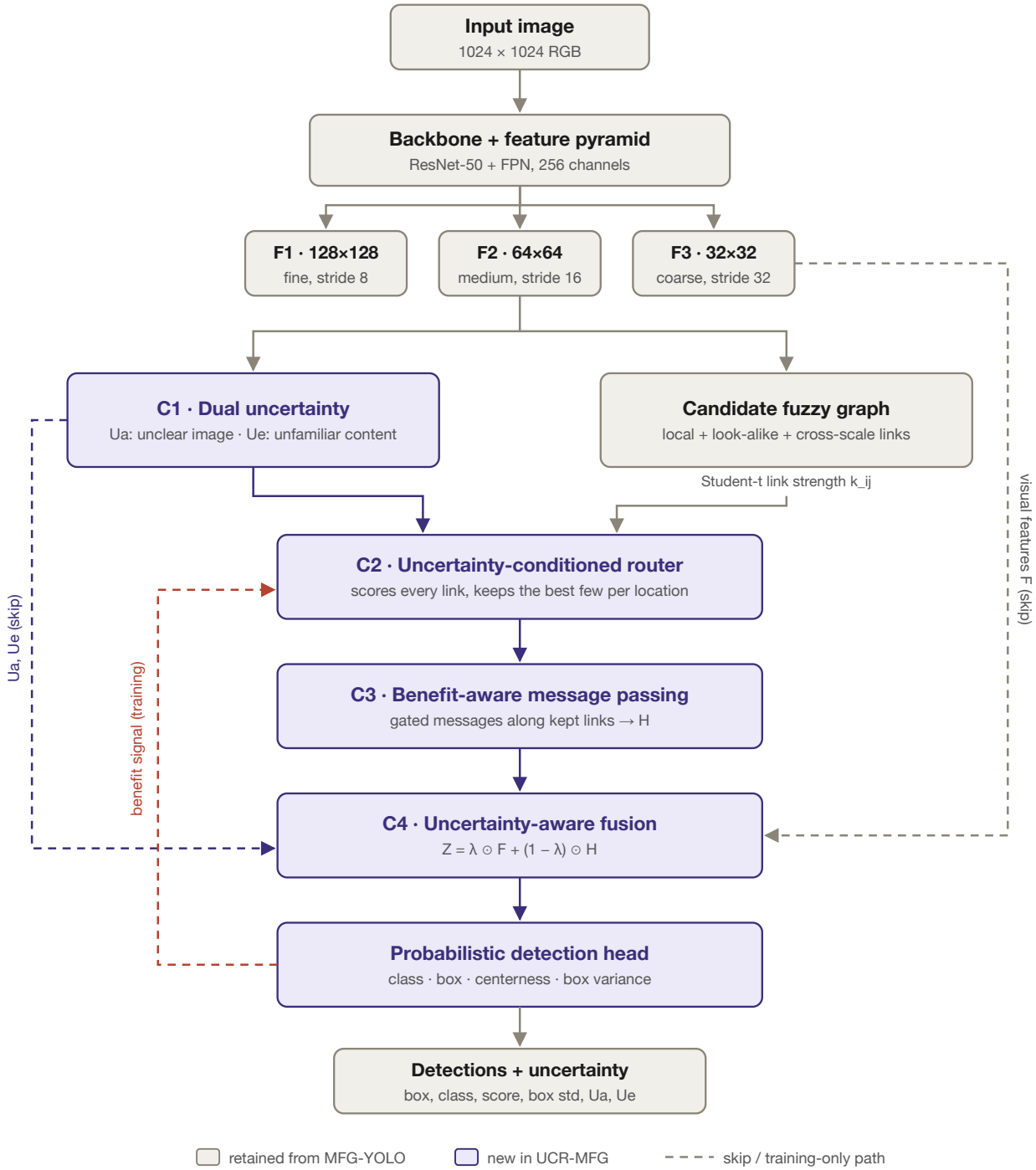


Figure 1. UCR-MFG architecture. The image flows from top to bottom. Gray blocks come from MFG-YOLO, purple blocks are the four contributions. Dashed lines carry the visual features and the uncertainty maps straight to the fusion gate, and (in red) the training-time benefit signal that teaches the router which links help.

Figure 1 shows the whole system. Before the equations, here is what it does in everyday terms. Think of a drone photo of a busy street. Some objects are easy: a bus in full view. Others are hard: a pedestrian half hidden behind a

tree, a blurry motorbike, a car that is only twelve pixels wide. A normal detector looks at each spot of the image mostly on its own and gives one confidence number. UCR-MFG does three extra things. It *notices* where it is unsure and why. It *asks the right neighbours* for help, instead of everyone. And it *decides how much to believe* that help before it makes the final call.

2.1 Following one image through the model

1. **Preparing the image.** The photo is resized and padded to 1024 × 1024 pixels without stretching it. Regions that VisDrone marks as "ignore" (crowds too dense to label, for example) are remembered, so the model is neither rewarded nor punished there.
2. **Seeing at three zoom levels.** The backbone turns the image into three feature maps. In the finest one, F1, each cell covers 8 × 8 pixels, which is what tiny pedestrians need. In the coarsest one, F3, each cell covers 32 × 32 pixels, which suits buses and trucks. Every cell holds a 256-number description of what it sees.
3. **Asking "how sure am I, and why?" (C1).** A small head looks at every cell and runs ten times with random parts switched off (dropout). If all ten runs agree but each one is hesitant, the image itself is unclear there: that is *aleatoric* uncertainty (blur, occlusion, shadow). If the ten runs disagree with each other, the model simply has not learned this kind of thing: that is *epistemic* uncertainty. Each cell gets both scores between 0 and 1.
4. **Drawing possible connections.** Every cell becomes a node in a graph. It is linked to its close neighbours, to cells elsewhere that look similar, and to the matching cells one zoom level up and down. Each link gets a fuzzy strength from a Student-*t* kernel: similar cells are strongly linked, and ambiguous pairs keep a weak link instead of being cut off. That is over a million possible links per batch, far too many to use them all.
5. **Choosing whom to ask (C2).** The router gives every link a usefulness score, based on what both cells look like, how strong their link is, how unsure each one is, and where they sit relative to each other. Each cell keeps only its best few links (about 31% of them survive). A cell that is unsure is allowed to keep a couple more, because it needs more help.
6. **Listening, but carefully (C3).** Along the kept links, neighbours send short summaries of what they see. Every message passes through a gate: a clear view of the same car passes almost fully, while a message from a shadow is turned down. Each cell then updates its own description with what it heard. The result is a second set of maps, H.
7. **Deciding how much to trust the neighbours (C4).** Now every cell has two descriptions: F (what it sees itself) and H (what its neighbours told it). A fusion gate mixes them channel by channel. For a clear object the gate stays near F; for an occluded or blurry one it leans more on H.
8. **Making the call, with error bars.** The detection head predicts the class, the box, a centerness score and how unsure it is about each side of the box. After removing duplicate boxes (NMS), every detection is reported with its score, box, box standard deviation in pixels, and its aleatoric and epistemic uncertainty.
9. **Learning which links help (training only).** Every second training step, the model checks every possible link and measures whether strengthening it would have lowered the detection error. The router is taught to pick the links that help the most. This is what makes the router benefit-aware rather than a plain similarity filter.

2.2 Where each part lives in the code

Part	File	What it does
Backbone + pyramid	`src/models/backbone.py`	ResNet-50 + FPN, produces F1, F2, F3

C1 Dual uncertainty	`src/models/uncertainty_head.py`	Aleatoric and epistemic maps from 10 dropout passes
Fuzzy graph	`src/models/graph_builder.py`	Candidate links, geometry, Student-t strengths
C2 Router	`src/models/router.py`	Link scores, uncertainty-based budgets, top-K selection
C3 Message passing	`src/models/message_passing.py`	Attention, benefit gate, GRU update, produces H
C4 Fusion	`src/models/fusion.py`	Learned mix of F and H
Probabilistic head	`src/models/detection_head.py`, `postprocess.py`	Class, box, box variance; NMS and per-detection uncertainty
Whole model	`src/models/ucr_mfg.py`	Connects the parts and switches them on stage by stage
Losses	`src/losses/*.py`	Detection, uncertainty, graph, router, consistency
Training and evaluation	`scripts/train.py`, `scripts/eval.py`, `src/engine.py`	Training loop, benefit pass, validation, metrics

3 Methodology

3.1 Overview and notation

> **In plain words**
>
> This section only fixes the symbols used below. F_l is the feature map at zoom level l; each of its cells is a graph node i. An arrow $(i \leftarrow j)$ means "node i listens to node j". U^a and U^e are the two uncertainty maps, H is the graph-enhanced map and Z is the fused map that the detector finally uses.

Figure 1 in Section 2 shows the full data flow.

An image $I \in \mathbb{R}^{3\times H\times W}$ is encoded into a pyramid $F_l \in \mathbb{R}^{C\times H_l\times W_l}$, $l \in \{1,2,3\}$, with strides $s_l \in \{8,16,32\}$ and $C = 256$. The current implementation uses a ResNet-50 feature pyramid; the method only requires three levels of width C. Each pyramid location is a graph node i with pixel center $\mathbf{p}_i = s_{l(i)}\,(x_i + \frac{1}{2},\ y_i + \frac{1}{2})$; the graph has $N = \sum_l H_l W_l$ nodes per image, and $f_i \in \mathbb{R}^C$ is the feature of node i. Edges are written $(i \leftarrow j)$: node i receives from node j. K is the number of classes, σ the logistic function, $\mathrm{sg}(\cdot)$ the stop-gradient operator and $1[\cdot]$ the indicator.

3.2 C1: Dual predictive uncertainty

> **In plain words: C1**
>
> **Goal.** Tell, for every spot in the image, how unsure the model is *and why*.
>
> **How.** A light head predicts a class guess and a box spread for every cell. To measure doubt, it is run ten times with random 20% of its inputs dropped. Imagine asking ten slightly different copies of the same expert. If they all say "maybe a car, maybe a van" in the same way, the picture itself is ambiguous (aleatoric). If one says car, another bus, another bicycle, the expert is out of its depth (epistemic). A wide predicted box spread at the centre of an object also adds to aleatoric uncertainty, because it means the edges of the object are hard to see.
>
> **Why it matters.** The two scores are not just reported. They steer the next three parts: who to ask (C2), how much to listen (C3) and how much to trust the answer (C4).

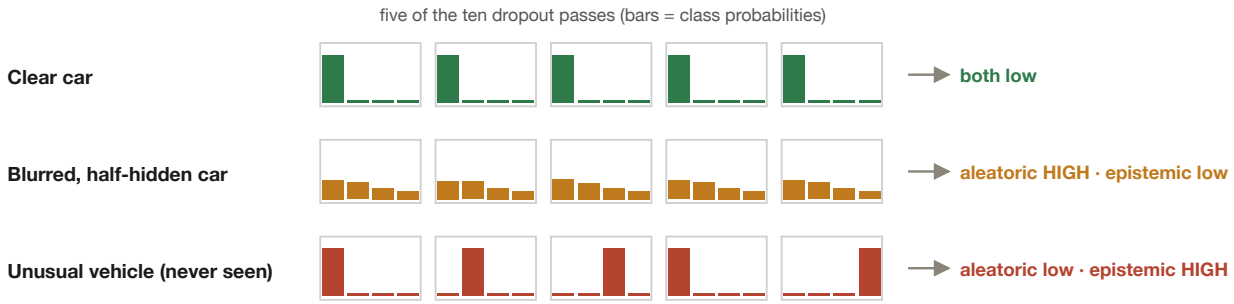


Figure 2. How ten dropout passes separate the two kinds of uncertainty. Agreeing but hesitant passes mean the image is unclear (aleatoric); disagreeing passes mean the model lacks knowledge (epistemic).

A shared tower ϕ (two 3×3 convolutions with GroupNorm and SiLU) feeds three 1×1 heads per location: box log-variances $\mathbf{u}_i \in \mathbb{R}^4$, class logits $\mathbf{z}_i \in \mathbb{R}^K$ and a pre-graph centerness $c_i^{\text{pre}} = \sigma(\mathbf{w}_c^\top \phi(F)_i)$.

Monte Carlo decomposition. With T stochastic passes under inverted dropout of rate p [10],

$$\mathbf{z}_i^{(t)} = W_z\, \phi\!\left(\frac{D^{(t)} \odot F}{1-p}\right)_i, \quad D^{(t)} \sim \text{Bernoulli}(1-p), \quad \mathbf{p}_i^{(t)} = \text{softmax}(\mathbf{z}_i^{(t)}), \quad \bar{\mathbf{p}}_i = \frac{1}{T}\sum_{t=1}^{T} \mathbf{p}_i^{(t)}. \tag{1}$$

With entropy $\mathcal{H}(\mathbf{p}) = -\sum_k p_k \log p_k$, the predictive entropy splits into an expected (aleatoric) term and the mutual information between prediction and weights (epistemic):

$$\underbrace{\mathcal{H}(\bar{\mathbf{p}}_i)}_{\text{total}} = \underbrace{\frac{1}{T}\sum_t \mathcal{H}(\mathbf{p}_i^{(t)})}_{\text{aleatoric } A_i} + \underbrace{\Big[\mathcal{H}(\bar{\mathbf{p}}_i) - \frac{1}{T}\sum_t \mathcal{H}(\mathbf{p}_i^{(t)})\Big]}_{\text{epistemic } \mathcal{I}_i}. \tag{2}$$

Localisation ambiguity enters through the predicted box variance, weighted by centerness so that off-center locations contribute little:

$$v_i = \frac{1}{4}\sum_{r=1}^{4} e^{u_{i,r}}, \qquad B_i = c_i^{\text{pre}}\, \frac{v_i}{\sigma_0^2 + v_i}. \tag{3}$$

$$U_i^a = \text{clip}_{[0,1]}\Big(\tfrac{1}{2}\,\frac{A_i}{\log K} + \tfrac{1}{2}\,B_i\Big), \qquad U_i^e = \text{clip}_{[0,1]}\Big(\frac{\max(\mathcal{I}_i, 0)}{\log K}\Big). \tag{4}$$

We use $T = 10$, $p = 0.2$, $\sigma_0^2 = 1$. The dropout masks of a batch are drawn from a generator seeded by (seed, epoch, batch index), so uncertainty maps are reproducible. When epistemic estimation is disabled (curriculum stage 1, Section 3.8) $A_i = \mathcal{H}(\text{softmax}\,\mathbf{z}_i)$ and $U_i^e = 0$. Both maps are computed without gradient and used as conditioning signals by C2–C4.

Training. On the positive set $\mathcal{P}$ (Section 3.7) with targets $t_{i,r}$ and the detector's distances $d_{i,r}$, the head is trained with a log-normal box likelihood and a class likelihood:

$$\mathcal{L}_u = \frac{1}{|\mathcal{P}|} \sum_{i \in \mathcal{P}} \left[\sum_{r=1}^{4} \left(\tfrac{1}{2} e^{-u_{i,r}} \left(\log t_{i,r} - \log \operatorname{sg}(d_{i,r}) \right)^2 + \tfrac{1}{2} u_{i,r} \right) - \log \operatorname{softmax}(\mathbf{z}_i)_{y_i} \right]. \tag{5}$$

The squared error is measured in log-distance space: it is scale-free across pyramid levels and stays bounded for an untrained detector, which a linear-space Gaussian likelihood does not.

3.3 Candidate multi-scale fuzzy graph

> **In plain words: the fuzzy graph**
>
> **Goal.** List every relationship that *might* be useful, before deciding which ones are.
>
> **How.** Each cell is linked to three kinds of partners: its immediate surroundings (a 5 × 5 window), up to eight look-alike cells in the same 32 × 32 neighbourhood of the map, and its "parent" and "children" at the zoom levels above and below. The cross-scale links let a tiny pedestrian borrow context from the coarser map, and let a large bus borrow detail from the finer one. Every link records direction and distance, and a fuzzy strength: identical-looking cells get a strength near 1, and very different ones a small but non-zero value, so nothing is thrown away too early.

Node embeddings are $\mathbf{h}_i = \mathrm{LN}(W_p f_i) \in \mathbb{R}^d$ with $d = 64$. Three candidate edge families are built per image:

- **Local**, same level: $\mathcal{E}_{\text{loc}} = \{(i \leftarrow j) : l(i) = l(j),\ 0 < \|(x_j - x_i,\ y_j - y_i)\|_\infty \le r\}$, $r = 2$.
- **Feature kNN**, same level: the $k_f = 8$ most cosine-similar nodes inside non-overlapping $B \times B$ spatial blocks ($B = 32$), keeping only neighbours outside the local window, so the families are disjoint.
- **Cross-scale**: each node i at level $l < 3$ has parent $\pi(i) = \left(\lfloor \mathbf{p}_{i,x} / s_{l+1} \rfloor, \lfloor \mathbf{p}_{i,y} / s_{l+1} \rfloor \right)$ at level $l + 1$. This gives top-down edges $(i \leftarrow \pi(i))$ and bottom-up edges $(\pi(i) \leftarrow i)$, so every fine node has one parent and every parent has up to four children.

Every edge carries geometry in receiver grid units and an edge-type index $\tau_{ij} \in \{\mathrm{loc}, \mathrm{knn}, \mathrm{td}, \mathrm{bu}\}$:

$$\boldsymbol{\delta}_{ij} = \frac{\mathbf{p}_j - \mathbf{p}_i}{s_{l(i)}}, \qquad \mathbf{g}_{ij} = \left[\frac{\boldsymbol{\delta}_{ij}}{1 + \|\boldsymbol{\delta}_{ij}\|},\ \log\left(1 + \|\boldsymbol{\delta}_{ij}\|\right) \right] \in \mathbb{R}^3. \tag{6}$$

Relational ambiguity is modeled by the heavy-tailed Student-t kernel of MFG-YOLO, normalised over each receiver's neighbourhood $\mathcal{N}(i)$:

$$k_{ij} = \left(1 + \frac{\|\mathbf{h}_i - \mathbf{h}_j\|^2}{\alpha} \right)^{-\frac{\alpha+1}{2}}, \qquad w_{ij} = \frac{\exp(k_{ij})}{\sum_{j' \in \mathcal{N}(i)} \exp(k_{ij'})}, \qquad \alpha = 1. \tag{7}$$

A Dirichlet energy regularises the embedding toward the fuzzy structure:

$$\mathcal{L}_{\text{graph}} = \frac{1}{|\mathcal{E}|} \sum_{(i \leftarrow j) \in \mathcal{E}} w_{ij} \, \|\mathbf{h}_i - \mathbf{h}_j\|^2. \tag{8}$$

Local and cross-scale edges depend only on the pyramid shape and are cached; only kNN edges are recomputed per image. A batch of b images is one disjoint graph with node offsets bN.

3.4 C2: Uncertainty-conditioned relational routing

> **In plain words: C2**
>
> **Goal.** Out of about a million possible links, keep only the ones that will actually help, and give more help to the locations that need it.
>
> **How.** A small network looks at each link (both cells' descriptions, the fuzzy strength, both cells' uncertainties, direction and distance, and link type) and gives it a score. Each cell then keeps its top 4 local links and top 2 of each other type. A cell's allowance grows by up to 2 per type when it is uncertain.
>
> **How it learns.** Two teachers. The first says "two cells inside the same object should usually talk". The second, and more important, says "this link actually lowered the detection error last time you tried it". That second signal comes from measuring, every other training step, how the error would change if each link were made stronger.

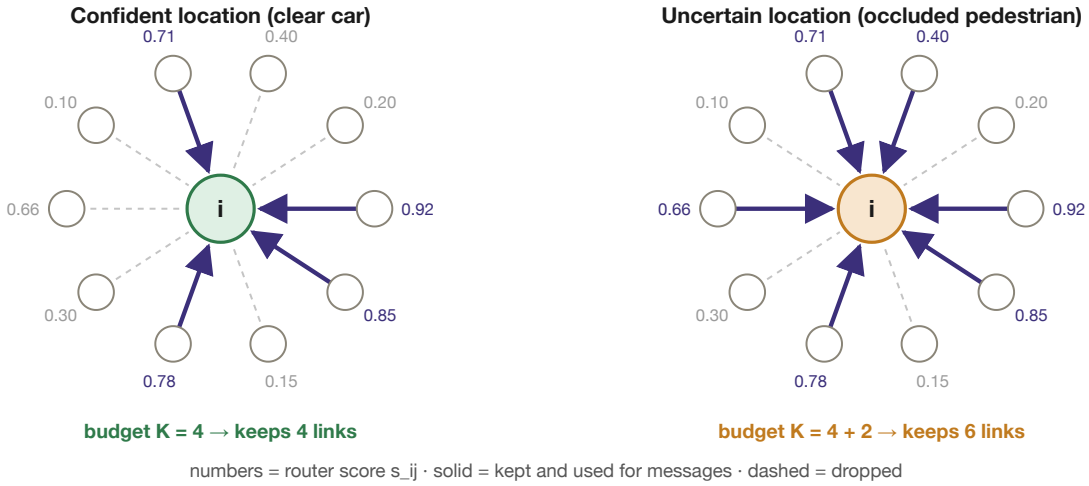


Figure 3. Routing at two locations. Both start with the same ten candidate links and the same scores, but the uncertain location is allowed to keep two more.

Edge utility. Each candidate is described by a 140-dimensional vector

$$\mathbf{x}_{ij} = \left[\mathbf{h}_i;\ \mathbf{h}_j;\ k_{ij};\ U_i^a;\ U_i^e;\ U_j^a;\ U_j^e;\ \mathbf{g}_{ij};\ \mathbf{e}_{\tau_{ij}}\right] \in \mathbb{R}^{64+64+1+4+3+4}, \tag{9}$$

with a learned type embedding $\mathbf{e}_\tau \in \mathbb{R}^4$, and scored by a two-layer MLP. Splitting the first layer $W_1 = [W_r, W_s, W_e]$ gives an exact factorisation in which the node terms are computed once per node rather than once per edge:

$$s_{ij} = \mathbf{w}_2^\top \operatorname{ReLU}\left(W_r\mathbf{h}_i + W_s\mathbf{h}_j + W_e\,\mathbf{x}_{ij}^{\text{edge}} + \mathbf{b}_1\right) + b_2. \tag{10}$$

Uncertainty-adaptive budgets. For receiver i and edge type τ with base budget K^τ ($K^{\text{loc}} = 4$, $K^{\text{knn}} = K^{\text{td}} = K^{\text{bu}} = 2$) and gain $\gamma = 2$:

$$K_i^\tau = K^\tau + \operatorname{round}\left(\gamma\,\max(U_i^a, U_i^e)\right), \qquad m_{ij}^{\text{hard}} = 1\left[\operatorname{rank}_{j' \in \mathcal{N}_\tau(i)}(s_{ij'})\big|_j < K_i^{\tau_{ij}}\right]. \tag{11}$$

Ambiguous locations therefore gather more context, while confident ones stay sparse. A straight-through estimator [12] keeps the binary forward selection differentiable:

$$m_{ij} = m_{ij}^{\text{hard}} + \sigma(s_{ij}) - \operatorname{sg}\left(\sigma(s_{ij})\right). \tag{12}$$

Algorithm 1. Vectorised per-receiver, per-type top-K selection

Input: scores $\mathbf{s} \in \mathbb{R}^E$, receivers $\mathbf{i}$, types $\boldsymbol{\tau}$, budgets $\mathbf{K} \in \mathbb{N}^E$. **Output:** $\mathbf{m}^{\text{hard}} \in \{0,1\}^E$.

1. Group id $g_e \leftarrow 4\, i_e + \tau_e$.
2. Score rank $\rho_e \leftarrow$ position of e in $\operatorname{argsort}(-\mathbf{s})$.
3. $o \leftarrow \operatorname{argsort}(g_e\, E + \rho_e)$ (groups contiguous; within a group, by descending score).
4. $c \leftarrow \operatorname{bincount}(g_o)$; start of each group $a \leftarrow \operatorname{cumsum}(c) - c$.
5. Within-group rank $q_n \leftarrow n - a_{g_{o_n}}$ for $n = 0, \dots, E-1$.
6. $m^{\text{hard}}_{o_n} \leftarrow 1[q_n < K_{o_n}]$. Cost $O(E \log E)$ with no Python loop over nodes.

Instance supervision. Let $o(i)$ be the smallest ground-truth box whose central region contains $\mathbf{p}_i$ ($|\mathbf{p}_i - \mathbf{c}_b| < \frac{\rho}{2}\,\mathbf{w}_b$ element-wise, $\rho = 0.5$), or $\varnothing$. On same-scale edges $\mathcal{E}_{\text{ss}}$,

$$y^{\text{inst}}_{ij} = 1\big[o(i) = o(j) \neq \varnothing\big], \qquad \mathcal{L}_{\text{inst}} = \frac{1}{|\mathcal{E}_{\text{ss}}|} \sum_{\mathcal{E}_{\text{ss}}} \text{BCE}_{\omega}\left(s_{ij}, y^{\text{inst}}_{ij}\right), \quad \omega = 3. \tag{13}$$

Benefit supervision. The router is taught which relations reduce the detection loss. Every $\Delta = 2$ steps a dense pass sends messages along *all* candidates (unselected ones have $m_{ij} = 0$ and no forward effect) and measures the first-order benefit of strengthening each edge:

$$\begin{gathered} b_{ij} = -\frac{\partial \mathcal{L}_{\text{det}}}{\partial m_{ij}}, \qquad y^{\text{ben}}_{ij} = 1\big[b_{ij} > 0 \ \wedge \ b_{ij} \geq Q_q(\{b > 0\})\big], \\ \mathcal{L}_{\text{benefit}} = \frac{1}{|\mathcal{E}|} \sum_{\mathcal{E}} \text{BCE}\left(s_{ij}, y^{\text{ben}}_{ij}\right), \end{gathered} \tag{14}$$

where Q_q is the $q = 0.75$ quantile of the positive benefits. Under mixed precision the gradient is taken on the loss-scaled $\mathcal{L}_{\text{det}}$; the labels depend only on ranks and signs, so they are unaffected by the scale.

3.5 C3: Benefit-aware multi-scale message passing

> **In plain words: C3**
>
> **Goal.** Let each cell learn from the neighbours the router picked, without being misled by bad advice.
>
> **How.** Every chosen neighbour sends a short message. Attention decides how the cell divides its listening between neighbours, and a separate gate (a number between 0 and 1) decides how much of each message to let through. The gate looks at who is listening, what the message says, how good the router thought the link was, and how uncertain both sides are. All accepted messages are added up and the cell updates its description with a GRU, a small memory unit that blends the old description with the news.
>
> **Efficiency.** Only kept links send messages, which is where the computing saving comes from. Everything that depends on a single cell, such as its message, is computed once per cell, not once per link.

Let $\mathcal{A}$ be the active edges: the routed edges $\{m^{\text{hard}}_{ij} = 1\}$ during normal steps and at inference, or all candidates during a benefit pass. Messages and attention scores are computed per node and gathered per edge:

$$\mathbf{v}_j = \text{MLP}(\mathbf{h}_j), \qquad e_{ij} = \text{LeakyReLU}_{0.2}\left(\mathbf{a}_r^{\top}\mathbf{h}_i + \mathbf{a}_s^{\top}\mathbf{h}_j\right). \tag{15}$$

The routing mask enters the attention normaliser, which is computed with a per-receiver maximum for float16 stability:

$$\alpha_{ij} = \frac{m_{ij}\, \exp\left(e_{ij} - \mu_i\right)}{\sum_{j' \in \mathcal{A}(i)} m_{ij'}\, \exp\left(e_{ij'} - \mu_i\right)}, \qquad \mu_i = \max_{j' \in \mathcal{A}(i)} e_{ij'}. \tag{16}$$

A benefit gate scales each message using the receiver state, the message content, the router score and both uncertainties:

$$\beta_{ij} = \sigma\big(\mathbf{q}_r^\top \mathbf{h}_i + \mathbf{q}_m^\top \mathbf{v}_j + \mathbf{q}_e^\top [s_{ij},\, U_i^a,\, U_i^e,\, U_j^e] + c\big). \tag{17}$$

$$\mathbf{a}_i = \sum_{j \in \mathcal{A}(i)} \beta_{ij}\, \alpha_{ij}\, \mathbf{v}_j, \qquad \mathbf{h}_i' = \mathrm{GRU}(\mathbf{a}_i, \mathbf{h}_i), \qquad H_i = \mathrm{LN}(W_o \mathbf{h}_i') \in \mathbb{R}^C. \tag{18}$$

H_i is reshaped into a pyramid $\{H_l\}$ aligned with $\{F_l\}$. Because m_{ij} appears only inside (16), an unselected edge still has a non-zero gradient, $\partial \alpha_{ij} / \partial m_{ij}\big|_{m_{ij}=0} = \exp(e_{ij} - \mu_i)/S_i$ with S_i the normaliser over the selected edges, which makes the benefit b_{ij} in (14) defined for every candidate. Multiplying the message by m_{ij} a second time would remove that gradient at $m_{ij} = 0$.

3.6 C4: Uncertainty-aware feature fusion

> **In plain words: C4**
>
> **Goal.** Combine "what I see" (F) with "what my neighbours told me" (H) in the right proportion for each location.
>
> **How.** A small network looks at both descriptions and both uncertainty scores and outputs a mixing weight λ for each of the 256 channels. $\lambda = 1$ means "trust my own eyes", $\lambda = 0$ means "trust the neighbours". It starts at about 0.88, so switching it on at the beginning of Stage 4 does not disturb a detector that already works, and it learns from there.

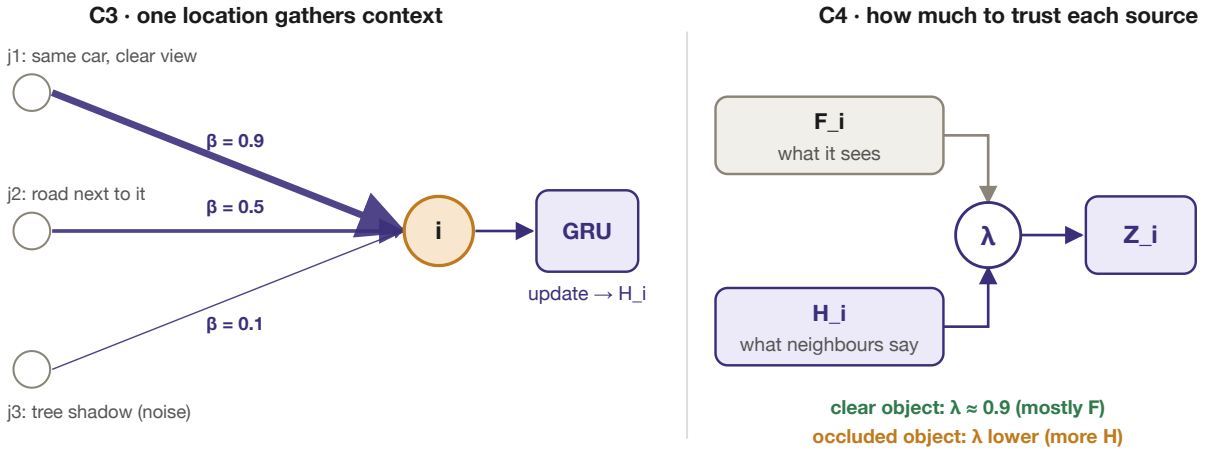


Figure 4. Left: C3 passes helpful messages strongly and suppresses noise (arrow width = gate value β). Right: C4 mixes the cell's own view F with its neighbours' view H.

$$\begin{aligned} \boldsymbol{\lambda}_i &= \sigma\Big(W_2\ \mathrm{SiLU}\ \big(\mathrm{GN}(W_1[F_i;\, H_i;\, U_i^a;\, U_i^e])\big) + \mathbf{b}_2\Big) \in (0,1)^C, \\ Z_i &= \boldsymbol{\lambda}_i \odot F_i + (1 - \boldsymbol{\lambda}_i) \odot H_i, \end{aligned} \tag{19}$$

with 1×1 convolutions W_1, W_2. The bias is initialised to $\mathbf{b}_2 = 2$, so $\boldsymbol{\lambda} \approx 0.88$ and fusion starts close to the visual features. The ablation baselines are $Z = \frac{1}{2}(F + H)$ and $Z = F + H$.

3.7 Probabilistic detection head

> **In plain words: the head**
>
> **Goal.** Turn the fused map into boxes that come with honest error bars.
>
> **How.** For every cell the head predicts which class is present, how far the box edges are to the left, top, right and bottom, how close the cell is to the object's centre, and how unsure it is about each edge. Training uses the standard FCOS recipe (focal loss for classes, CIoU for boxes) plus a likelihood term that teaches the edge uncertainty to match the real error. At test time, every detection is reported with its box spread in pixels and with C1's two uncertainty scores at its location.

An anchor-free FCOS head [7] with separate classification and regression towers (four 3×3 convolutions each) predicts, per location, class logits $\mathbf{q}_i$, stride-normalised distances $\mathbf{d}_i = \exp(\kappa_l\, \mathbf{r}_i) \in \mathbb{R}^4_+$ to the left, top, right and bottom box sides (learnable level scale κ_l), a centerness logit $\hat{c}_i$ and box log-variances $\hat{\mathbf{u}}_i \in \mathbb{R}^4$.

Assignment. A location is positive for box b if it lies inside b, within $1.5\, s_l$ of its center, and $\max_r t_{i,r}\, s_l$ falls in the level range $[0, 64], [64, 128], [128, \infty)$ pixels. Ties go to the smallest box, and locations inside annotated ignore regions are excluded. The centerness target is $c^\star_i = \sqrt{\frac{\min(l,r)}{\max(l,r)} \cdot \frac{\min(t,b)}{\max(t,b)}}$.

$$\mathcal{L}_{\text{det}} = \frac{1}{|\mathcal{P}|}\Big[\sum_i \mathrm{FL}(\mathbf{q}_i, y_i) + \sum_{i \in \mathcal{P}} c^\star_i\, \mathcal{L}_{\text{CIoU}}(\hat{b}_i, b_i) + \sum_{i \in \mathcal{P}} \mathrm{BCE}(\hat{c}_i, c^\star_i)\Big], \tag{20}$$

$$\mathcal{L}_{\text{var}} = \frac{1}{|\mathcal{P}|} \sum_{i \in \mathcal{P}} \sum_{r=1}^{4} \Big(\tfrac{1}{2} e^{-\hat{u}_{i,r}} \big(\log t_{i,r} - \log \mathrm{sg}(d_{i,r}) \big)^2 + \tfrac{1}{2} \hat{u}_{i,r} \Big), \tag{21}$$

with sigmoid focal loss FL ($\gamma = 2$, $\alpha = 0.25$) [8] and Complete-IoU loss [9]. Detaching the mean in (21) lets the variance model the head's residual error without moving the boxes.

Inference. Scores are $\varsigma_{ik} = \sqrt{\sigma(q_{ik})\, \sigma(\hat{c}_i)}$. Candidates with $\varsigma > 0.05$ (top 1000 per level) are decoded to boxes, filtered by class-wise NMS (IoU 0.6) and capped at 500 per image. Each detection carries its location's U^a_i, U^e_i and a per-side one-sigma box deviation in pixels:

$$\sigma^{\text{px}}_{i,r} = s_l\, d_{i,r}\, \big(e^{\hat{u}_{i,r}/2} - 1 \big). \tag{22}$$

3.8 Cross-scale consistency, total objective and curriculum

> **In plain words: training**
>
> **Cross-scale consistency.** The class guess for an object at one zoom level should roughly agree with the guess one level coarser. A small penalty encourages that.
>
> **Why stages.** Switching everything on from the first epoch would let random graph features confuse a detector that cannot yet detect anything. So training grows the model in four stages: first a solid detector with uncertainty, then the graph, then routing and message passing (with the graph features added at only 10% strength, so the router gets feedback without destabilising things), and finally the learned fusion gate.



Figure 5. The four-stage training curriculum over 150 epochs.

For each ground-truth center, let $\mathbf{q}^{(l)}$ be the logits at its cell on level l, using the smallest box when several share a cell. Predictions are aligned across adjacent levels with the coarser level as a detached teacher:

$$\mathcal{L}_{\text{cons}} = \frac{1}{|\mathcal{S}|} \sum_{(l,\text{img})\in\mathcal{S}} \frac{1}{n} \sum_{\text{objects}} \mathrm{KL}\left(\operatorname{softmax}\left(\operatorname{sg}(\mathbf{q}^{(l+1)}) \right) \middle\| \operatorname{softmax}\left(\mathbf{q}^{(l)} \right) \right). \tag{23}$$

The training objective at curriculum stage $\kappa \in \{1,2,3,4\}$ is

$$\begin{aligned} \mathcal{L} = \mathcal{L}_{\text{det}} + \lambda_{\text{var}}\mathcal{L}_{\text{var}} + \lambda_u\mathcal{L}_u + 1[\kappa \geq 2]\,\lambda_g\mathcal{L}_{\text{graph}} \\ + 1[\kappa \geq 3]\left(\lambda_{\text{inst}}\mathcal{L}_{\text{inst}} + \lambda_{\text{ben}}\mathcal{L}_{\text{benefit}}\right) + 1[\kappa = 4]\,\lambda_c\mathcal{L}_{\text{cons}}, \end{aligned} \tag{24}$$

with $\lambda_{\text{var}} = \lambda_u = 0.5$, $\lambda_g = 0.1$, $\lambda_{\text{inst}} = \lambda_{\text{ben}} = 0.2$, $\lambda_c = 0.1$; $\mathcal{L}_{\text{benefit}}$ is active only on benefit steps.

Stage	Epochs	Input to the head	Active losses
1	1–30	F (no Monte Carlo passes)	det, var, u
2	31–60	F; graph built, U^e estimated	+ graph
3	61–90	$F + 0.1\,H$ (C2 and C3 receive a detection signal)	+ inst, benefit
4	91–150	Z from the C4 gate (19)	+ consistency

Table 1. Curriculum. At inference the model runs at its checkpoint stage with $T = 10$ Monte Carlo passes.

Algorithm 2. One UCR-MFG training step (stage κ, global step n)

1. $\{F_l\} \leftarrow \text{Backbone}(I)$; $\{U_l^a, U_l^e\} \leftarrow$ C1 (1)–(4), Monte Carlo only if $\kappa \geq 2$.
2. **if** $\kappa \geq 2$: build $\mathcal{G} = (\mathbf{h}, \mathcal{E}, \mathbf{g}, k, w)$ by (6)–(7).
3. **if** $\kappa \geq 3$:
   a. s_{ij} by (9)–(10); m^{hard} by (11) with Algorithm 1; m_{ij} by (12).
   b. dense $\leftarrow (n \bmod \Delta = 0)$; $\mathcal{A} \leftarrow \mathcal{E}$ if dense, else $\{m^{\text{hard}} = 1\}$.
   c. $H \leftarrow$ C3 (15)–(18) over $\mathcal{A}$; $Z \leftarrow F + 0.1H$ if $\kappa = 3$, else C4 (19).
4. Head on Z (or F); assign targets; compute every term of (24) except $\mathcal{L}_{\text{benefit}}$.
5. **if** dense: $b \leftarrow -\nabla_m \mathcal{L}_{\text{det}}$ (graph retained); add $\lambda_{\text{ben}}\mathcal{L}_{\text{benefit}}(s, y^{\text{ben}}(b))$ by (14).
6. Back-propagate $\mathcal{L}$ with float16 loss scaling; every 2 steps clip $\|\nabla\|_2 \leq 10$ and take an AdamW step.

Optimisation. AdamW (learning rate 10^{-4}, weight decay 10^{-4}) with cosine decay to 10^{-6} over 150 epochs, batch size 2 with 2-step gradient accumulation, float16 autocast, 1024×1024 input. Validation mAP is computed every 5 epochs and the best model is kept.

3.9 Complexity

> **In plain words**
>
> The router is what keeps the cost under control: the graph proposes about 1.27 million links for a batch of two images, but only about 390 thousand carry messages. On a 16 GB Apple-silicon Mac, one full Stage-4 training step takes about 2 seconds.

Per image the candidate set has $|\mathcal{E}| \approx \sum_l H_l W_l \left[(2r+1)^2 - 1\right] + |\mathcal{E}_{\text{knn}}| + 2\sum_{l<3} H_l W_l$ edges, but messages and gates are evaluated only on $\sum_{i,\tau} K_i^\tau$ routed edges, and every per-node quantity in (10) and (15)–(17) is computed once per node. At 1024^2 with $r = 2$, a batch of two images has about 1.27×10^6 candidates, of which about 3.9×10^5 (31%) are routed.

3.10 Evaluation measures

> **In plain words**
>
> **mAP** answers "how many objects did it find, and how tight are the boxes?", averaged over strict and loose overlap thresholds. **D-ECE** answers "when the model says 80% sure, is it right about 80% of the time?". The **AUROC** scores answer "do high uncertainty values actually point at the wrong detections?"; 0.5 means no better than chance, and 1.0 means perfect.

Detection accuracy is COCO-style AP averaged over IoU thresholds $\{0.50, 0.55, \dots, 0.95\}$ with 101-point interpolated precision. We also report AP50, AP75 and AP for small ($< 32^2$ px), medium and large ($> 96^2$ px) objects. Unmatched detections centered in VisDrone ignore regions are discarded. Calibration is measured by the detection expected calibration error over $M = 15$ confidence bins, counting duplicates and background detections as errors:

$$\mathrm{D\text{-}ECE} = \sum_{m=1}^{M} \frac{|\mathcal{B}_m|}{n} \left| \mathrm{acc}(\mathcal{B}_m) - \mathrm{conf}(\mathcal{B}_m) \right|. \tag{25}$$

Uncertainty quality is measured as the AUROC of each of U^a, U^e, $\bar{\sigma}^{\mathrm{px}}$ and $1 - \varsigma$ for separating false from true detections (true: same class and IoU ≥ 0.5).

4 Working with the project

This section is a practical guide: how to set the project up, train it, check it and run the experiments the paper needs. All commands are run from the project folder.

4.1 Setup and data

```
pip install -r requirements.txt
pytest tests/                       # 36 automated tests
```

Place the datasets under `dataset/` as described in the README: VisDrone2019-DET is the training set, UAVDT is only used for zero-shot testing, and DOTA-v1.5 and SODA-A are optional separate tracks. Before the first training run, audit the data:

```
python scripts/validate_data.py --config configs/datasets/visdrone.yaml --split val --samples 548
```

4.2 Training

```
python scripts/train.py --config configs/experiments/visdrone_primary.yaml
python scripts/train.py --config configs/experiments/visdrone_primary.yaml --resume latest   # af-
ter Ctrl+C or a crash
```

- The four stages switch automatically by epoch (Figure 5). The progress bar shows the losses; `L_det` should fall steadily.
- Every 5 epochs the model is scored on VisDrone val. The best model is saved as `checkpoints/visdrone_primary/best.pth`, the newest as `latest.pth`.
- Pressing Ctrl+C saves a checkpoint safely; `--resume latest` continues from it with the optimiser, learning rate and random state restored.
- Memory: with the default local window `r_local: 2` a 16 GB Mac stays at about 11–12 GB. The value 3 doubles the number of links and needs a larger GPU.

4.3 Evaluation

```
python scripts/eval.py --config configs/experiments/visdrone_primary.yaml \
    --weights checkpoints/visdrone_primary/best.pth --split test_dev --output visdrone_testdev.json
python scripts/eval.py --config configs/experiments/uavdt_zeroshot.yaml \
    --weights checkpoints/visdrone_primary/best.pth --split test
```

The script rebuilds the model from the configuration saved inside the checkpoint and runs it at the stage it was trained to. It prints mAP, AP50, AP75, AP for small, medium and large objects, D-ECE, the uncertainty AUROC values and per-class AP. For UAVDT, VisDrone's car and van are mapped to car, truck to truck, and bus to bus.

4.4 Ablations: proving each contribution

Each contribution can be switched off in `configs/model/default.yaml`. Train one model per row with the same seed and schedule, then compare them with `eval.py`.

Setting	What it removes	Question it answers
`router.enabled: false`	Routing (every candidate link is used)	Does selecting links beat using all of them, as in MFG-YOLO?
`router.use_uncertainty: false`	Uncertainty in the router's scores and budgets	Is routing better because it is *uncertainty-conditioned*?
`router.k_gain: 0`	Larger budgets for uncertain locations	Does giving uncertain locations more links help?
`mpnn.benefit_gate: false`	The message gate ($\beta_{ij} = 1$)	Does weighing each message help C3?
`fusion.mode: fixed` / `add`	The learned fusion gate	Does reliability-based fusion beat a fixed mix?
`uncertainty.mc_passes: 1`	Epistemic uncertainty	Is the dual (two-part) uncertainty needed?

4.5 Checking the router

```
python scripts/oracle_router_pilot.py --weights checkpoints/visdrone_primary/best.pth --samples 50
```

For each link type, this reports how many candidate links connect two points of the same object, how many of the router's chosen links do (its precision), the best possible precision with the same number of links (the oracle), and how many of the useful links the router found (recall). A trained router should sit well above the base rate and close to the oracle.

References

1. N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, S. Zagoruyko, “End-to-end object detection with transformers,” *ECCV*, 2020, pp. 213–229.
2. X. Zhu, W. Su, L. Lu, B. Li, X. Wang, J. Dai, “Deformable DETR: Deformable transformers for end-to-end object detection,” *ICLR*, 2021.
3. G. Zhao, W. Ge, Y. Yu, “GraphFPN: Graph feature pyramid network for object detection,” *ICCV*, 2021, pp. 2763–2772.
4. J. Mukhoti, A. Kirsch, J. van Amersfoort, P. H. S. Torr, Y. Gal, “Deep deterministic uncertainty: A new simple baseline,” *CVPR*, 2023, pp. 24384–24394.
5. Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, J. Chen, “DETRs beat YOLOs on real-time object detection,” *CVPR*, 2024, pp. 16965–16974.
6. T. Xie, Q. Wu, T. Yin, M. Gao, W. Ding, W. Pedrycz, “MFG-YOLO: Multi-scale fuzzy graph representation learning for uncertainty-aware object detection,” *Expert Systems with Applications*, vol. 333, art. 134171, 2027, doi:10.1016/j.eswa.2026.134171.
7. Z. Tian, C. Shen, H. Chen, T. He, “FCOS: Fully convolutional one-stage object detection,” *ICCV*, 2019, pp. 9627–9636.
8. T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, “Focal loss for dense object detection,” *ICCV*, 2017, pp. 2980–2988.
9. Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, D. Ren, “Distance-IoU loss: Faster and better learning for bounding box regression,” *AAAI*, 2020, pp. 12993–13000.
10. Y. Gal, Z. Ghahramani, “Dropout as a Bayesian approximation: Representing model uncertainty in deep learning,” *ICML*, 2016, pp. 1050–1059.
11. A. Kendall, Y. Gal, “What uncertainties do we need in Bayesian deep learning for computer vision?,” *NeurIPS*, 2017.
12. Y. Bengio, N. Léonard, A. Courville, “Estimating or propagating gradients through stochastic neurons for conditional computation,” arXiv:1308.3432, 2013.